

Embedded Ray-Traced Propagation for End-to-End 6G Simulation in ns-3

Aung Kaung Myat^{a,*}, Hrishikesh Dutta^a, Roberto Minerva^a, Djamal Zeghlache^a and Noel Crespi^a

^a*SAMOVAR, Télécom SudParis, Institut Polytechnique de Paris, , Palaiseau, France*

ARTICLE INFO

Keywords:

ns-3
Sionna RT
ray tracing
5G NR
massive MIMO
energy modeling
propagation cache
network simulation

ABSTRACT

Stochastic propagation models in packet-level 5G simulators struggle to reproduce site-specific multipath effects, while deterministic model based on ray tracing is computationally expensive for realistic simulation rates. To bridge this gap, we introduce **sionnart**, an integrated simulation framework that embeds the Sionna RT ray tracing engine directly into the ns-3 process via pybind11, exposing deterministic channel information to standard ns-3 propagation interfaces without the overhead of inter-process communication. To make ray tracing practical for mobile user equipment (UE), our framework employs two novel mechanisms: a displacement-based coherence cache that reuses channel parameters within a geometry-dependent validity radius, and an adaptive virtual receiver generation technique that proactively pre-computes near-future UE positions during a single batched Sionna RT query. In our framework performance benchmarking involving 32 UEs, **sionnart** outperform the standard socket-coupled **ns3sionna** baseline by a factor of 43.8 under high mobility, all while keeping the GPU memory footprint below 500 MiB. As a practical application case study, we demonstrate the framework's utility in a massive MIMO 5G deployment with 20 UEs and various gNB array sizes across free-space, urban macro, and urban micro environments. This case study reveals that deterministic multipath significantly alters critical metrics like CQI, MCS selection, throughput, and gNB energy consumption compared to stochastic models, particularly in geometry-sensitive deployments. These results confirm that our integrated simulation tool successfully makes repeatable, geometry-aware system-level evaluation both computationally viable and highly practical.

1. Introduction

Sixth-generation (6G) cellular systems are expected to operate in environments where the radio channel is shaped by geometry, blockage, material properties, mobility, and highly directional antenna arrays [1, 2, 3]. Techniques like integrated sensing and communication (ISAC) and AI-assisted beam management all depend on spatial information such as angles, delays, blockage states, and multipath structure [4, 5, 6]. Validating these systems in real-world deployments is costly and difficult to reproduce. Thus, simulation that captures site-specific propagation geometry is therefore the best alternative to test these techniques [6, 7].

This creates a methodological gap for network research. System-level simulators are needed because the quantities that users observe – throughput, delay, jitter, packet loss, scheduling decisions, retransmissions, and energy consumption. ns-3 provides this discrete-event protocol-stack capability through a mature C++ framework and, through 5G-LENA, a detailed 5G NR

stack with scheduling, link adaptation, and MIMO support [8, 9]. However, the propagation models commonly used in ns-3, including Friis, log-distance, and standardized stochastic models such as 3GPP TR 38.901, are designed to be computationally efficient abstractions [1, 9]. They are valuable for average-case studies, but they do not reproduce the site-specific propagation paths created by a particular street canyon, indoor layout, or deployment geometry.

Ray tracing addresses this missing physical realism by computing propagation paths directly from a three-dimensional scene [2, 3, 10]. Sionna RT provides a modern GPU-accelerated ray tracing engine with support for antenna arrays, polarization, and scene-based propagation studies [11, 12]. Yet Sionna RT is not a network simulator: it does not provide the NR protocol stack, packet scheduler, traffic models, mobility procedures, or energy accounting required to study end-to-end network behavior [11, 12, 9]. The main issue is therefore not whether ray tracing or protocol simulation is useful in isolation, but whether deterministic ray-traced channels can be made practical for packet-level ns-3 simulations.

Existing integrations typically connect ns-3 to a separate Python ray tracing process through sockets. The **ns3sionna** implementation follows this design and exchanges channel information with Sionna RT over ZMQ [13]. This separation is flexible, but it introduces

*Corresponding author

 aung-kaung.myat@telecom-sudparis.eu (A.K. Myat);
hrishikesh.dutta@telecom-sudparis.eu (H. Dutta);
roberto.minerva@telecom-sudparis.eu (R. Minerva);
djamal.zeghlache@telecom-sudparis.eu (D. Zeghlache);
noel.crespi@telecom-sudparis.eu (N. Crespi)
ORCID(s):

per-query inter-process communication, serialization, and duplicated state management. Those costs become especially visible under mobility, where small position changes repeatedly invalidate channel samples while the simulator continues to issue packet-level propagation queries. In this work, we take the opposite design choice: Sionna RT is embedded directly into the ns-3 process, and deterministic channel information is exposed through standard ns-3 propagation interfaces.

This paper presents **sionnart**, an extension module of ns-3 that provides a fast, repeatable, and ray-traced wireless network simulation environment. The primary goal is to preserve ns-3 and 5G-LENA as the reliable source of protocol behavior, while seamlessly substituting purely analytical propagation models with site-specific channel information from Sionna RT. Consequently, our research addresses two critical questions: first, how can we integrate ns-3 and Sionna RT to reduce the computational overhead of ray tracing to a level practical for packet-level simulation? Second, when applied to a real-world case study, how does this deterministic multipath data alter 5G NR performance and energy conclusions compared to conventional ns-3 propagation models?

The main contributions of this work are:

1. **An embedded ns-3-Sionna RT simulation framework.** We implement **sionnart**, integrating the Sionna RT engine directly into the ns-3 address space via pybind11 [11, 12, 14]. This architecture connects to ns-3 through `PropagationLossModel`, `PropagationDelayModel`, and spectrum propagation logic. It enables packet-level simulations to seamlessly request ray-traced path loss, delay, and MIMO channel frequency responses without relying on slow socket-level round trips.
2. **Computational-efficiency mechanisms for practical execution.** To make ray tracing viable under mobility, our framework introduces a displacement-based coherence cache, whose validity radius adjusts based on link distance and transmit array size, coupled with an adaptive virtual receiver generation technique. The cache efficiently reuses path loss, delay, LoS state, and MIMO channel information while nodes remain within their spatial validity radius. Meanwhile, the virtual receiver mechanism proactively predicts short-horizon UE positions and bundles them into the same Sionna RT batched call, transforming reactive cache misses into proactive look-aheads without any added ray tracing overhead.
3. **Framework Benchmarking and Application Case Study.** We evaluate the framework's performance by benchmarking **sionnart** against pure ns-3 propagation and the socket-coupled **ns3sionna** baseline in IEEE 802.11ax scenarios.

At $N = 32$, **sionnart** operates up to $232.3\times$ faster than **ns3sionna** in stationary high-load conditions, and $43.8\times$ faster under high mobility, keeping GPU memory usage comfortably below 500 MiB. Finally, we present an application case study using the embedded backend in a 5G NR deployment with 20 UEs, 2×2 , 4×4 , and 8×8 gNB arrays across free-space, urban macro, and urban micro radio environments [9, 1, 7]. This practical demonstration highlights how deterministic multipath consistently shifts CQI/MCS selection, application throughput, packet loss, and gNB energy trends relative to stochastic models, particularly in geometry-sensitive urban deployments.

The remainder of this paper is organized as follows. Section 2 provides background on ns-3, 5G-LENA, Sionna RT, and ray-traced propagation. Section 3 reviews prior integration approaches, including stochastic channel models in NR simulators and socket-coupled ray tracing frameworks. Section 4 details the embedded architecture, displacement-based cache, adaptive virtual receiver generation, propagation calculations, and the NR energy formulation. Section 5 benchmarks the runtime scaling and GPU footprint of our framework against pure ns-3 and **ns3sionna**. Section 6 presents the application case study, evaluating the impact of deterministic ray-traced propagation on NR application, propagation, and energy metrics. Section 7 concludes the paper and discusses future work toward ISAC-oriented simulation.

2. Background

This section introduces the three components that the proposed framework builds upon: ns-3 with 5G-LENA as the protocol-level simulation environment, Sionna RT as the ray tracing engine, and ray-traced propagation as the physical modeling approach. Related work on prior integration approaches and alternative channel modeling strategies is discussed in Section 3.

2.1. Ray-Traced Propagation

Ray tracing models radio propagation by computing geometric paths between transmitters and receivers in a three-dimensional scene [2, 3]. Each ray undergoes reflections, diffractions, and scattering at surfaces, yielding multipath components with physically derived delays, angles of arrival and departure, and complex amplitudes. In contrast to stochastic models, which represent path loss and multipath through statistical distributions and cannot distinguish a line-of-sight path along an open street from one blocked by a building, ray tracing produces a site-specific channel that evolves smoothly and predictably as a node moves through the environment [2, 1].

This type of propagation model is especially valuable for 6G research, where mmWave and THz propagation is highly directional and sensitive to deployment geometry, and where applications such as ISAC require channel information that reflects the physical structure of the scene [19, 3, 4]. Although ray tracing is computationally demanding, GPU-accelerated implementations have made it practical for simulation-scale studies [12, 10, 6].

2.2. ns-3 and 5G-LENA

ns-3 is an open-source discrete-event network simulator written in C++, widely used for protocol-level research [8]. Its core abstraction mirrors real network hardware: a `Node` aggregates a `NetDevice`, protocol stack, and `Application`, while a `Channel` connects devices and invokes a `PropagationLossModel` and `PropagationDelayModel` on each packet transmission. The 5G-LENA module extends ns-3 with a 3GPP-compliant 5G NR stack, implementing PHY, MAC, RLC, and PDCP layers with flexible OFDMA schedulers, link adaptation, and MIMO beamforming [9, 7, 18].

Both ns-3 and 5G-LENA rely on analytical or stochastic propagation models such as Friis, log-distance, or 3GPP TR 38.901, which represent path loss and multipath through statistical distributions [8, 1, 9]. These abstractions are efficient for large-scale average-case studies but do not capture site-specific propagation paths, spatially consistent channel evolution under mobility, or the geometric multipath structure required for ISAC and beamforming research.

2.3. Sionna RT

Sionna RT is a GPU-accelerated, differentiable ray tracing engine for radio propagation modeling, built on Mitsuba 3 and Dr.JIT with a PyTorch API [11, 12]. It resolves multipath components for arbitrary antenna arrays with support for polarization and configurable material properties, and computes channel impulse responses for entire scenes in a single differentiable forward pass.

Sionna RT does not include a network protocol stack and it does not support NR scheduling, link adaptation, packet queuing, mobility procedures, or energy accounting [11, 12]. This makes it a natural complement to ns-3 with 5G-LENA, where Sionna RT supplies the physical channel and ns-3 supplies the protocol behavior. To integrate the two, we leverage pybind11 to embed the Python interpreter and Sionna RT engine directly within the ns-3 process [14], replacing stochastic propagation abstractions with deterministic, environment-specific channel information.

3. Related Work

3.1. Stochastic Channel Models in NR Simulation

The 3GPP TR 38.901 stochastic model is the de facto standard for system-level NR evaluation, providing path-loss, shadowing, and multipath cluster models for urban macro, urban micro, and indoor environments across sub-6 GHz and mmWave bands [1]. This model is integrated directly into ns-3 and 5G-LENA, enabling large-scale multi-cell studies without a geometric scene description [8, 9].

Stochastic models are well suited for average-case performance analysis but generate independent channel realizations per link and per simulation run. They cannot reproduce the spatial correlation of multipath components as a user moves through a specific deployment, nor can they resolve the site-specific angles and delays required for evaluating ISAC or spatially sensitive beamforming techniques [2, 3, 4]. This limitation motivates the use of deterministic propagation backends in protocol-level simulation.

3.2. Integrating Ray Tracing with Network Simulator

Coupling ray tracing engines with network simulators has been explored in several prior works. The `ns3sionna` framework connects ns-3 to Sionna RT through a ZMQ socket interface, exchanging channel data between separate processes [13]. While this loose coupling preserves the independence of both tools, it introduces per-query inter-process communication and serialization overhead that grows with station count and update frequency under mobility.

Prior to the present work, we investigated coupling Sionna RT with Simu5G, a 5G system-level simulator built on the OMNeT++ framework [16, 15]. The OMNeT++ architecture, which uses NED-defined topologies instantiated through a C++ runtime, introduces integration complexity when connecting to Python-based tools [15, 16]. Additionally, Simu5G does not expose a native beamforming interface for MIMO antenna arrays at the time of writing, requiring workarounds that add brittleness to the simulation pipeline [16]. These constraints, combined with the stochastic propagation models used by default, motivated the transition to ns-3 with 5G-LENA, which provides native MIMO support, a Python-accessible C++ API, and a flexible propagation interface that can be replaced without modifying the protocol stack.

The approach presented in this paper differs from prior couplings in three respects. First, Sionna RT is embedded directly within the ns-3 process via pybind11, eliminating inter-process communication and serialization overhead entirely [14]. Second, a displacement-based coherence cache avoids redundant ray tracing calls when nodes have not moved beyond the spatial validity radius of a prior result, reducing

per-packet query overhead without sacrificing channel accuracy. Third, adaptive virtual receiver generation proactively pre-computes channel samples for predicted near-future UE positions in the same Sionna RT call, converting reactive cache misses into batched look-aheads and further reducing the number of ray tracing invocations under mobility. Together, these mechanisms make deterministic ray-traced propagation practical as a live packet-level simulation backend, which no prior integration has demonstrated at the protocol-stack fidelity provided by ns-3 with 5G-LENA.

4. Methodology

The overall architecture of the proposed integrated simulation framework is depicted in Figure 1. The proposed integrated simulation framework follows a layered design in which ns-3 remains the primary discrete-event simulator while Sionna RT is embedded as an environment-aware ray tracing engine [14, 13, 11, 12]. The architecture preserves the mature protocol, traffic generation, and scheduling capabilities of ns-3 and Sionna RT handles ray tracing to provide a site-specific propagation channel model, with GPU acceleration for computationally intensive operations.

The application layer generates user-defined traffic between UEs and the gNB. Traffic types include video streaming, social media, gaming, FTP, and VoIP, enabling simulation of diverse workload scenarios.

The middle layer (shown in green) contains the ns-3 network stack, which handles node management, mobility, routing, scheduling, and MAC operations for the chosen protocol standard (Wi-Fi 802.11ax or 5G NR). When a packet is generated, it passes through the protocol stack until it reaches the wireless medium, at which point ns-3 invokes `PropagationLossModel` and `PropagationDelayModel` to obtain path loss, transmission delay, and channel response.

When `PropagationLossModel` or `PropagationDelayModel` is invoked, the Sionna RT engine computes environment-aware channel parameters (path loss, propagation delay, and arrival and departure angles) for the requested transmitter–receiver pair. Issuing a full ray tracing query on every packet event is computationally expensive and would dominate simulation runtime. The framework addresses this through two complementary mechanisms: a coherence-distance cache that reuses recently computed parameters for spatially close positions, and adaptive virtual receiver pre-computation that pre-populates the cache with channel parameters for predicted future node positions. On each query for a link (p_{tx}, p_{rx}) , the cache layer computes the spatial displacement between the current positions and those stored in the cache entry. If the displacement is below the coherence distance d_{coh} , the cached parameters are returned directly. Otherwise, the Sionna RT engine is invoked and the cache is updated with the newly

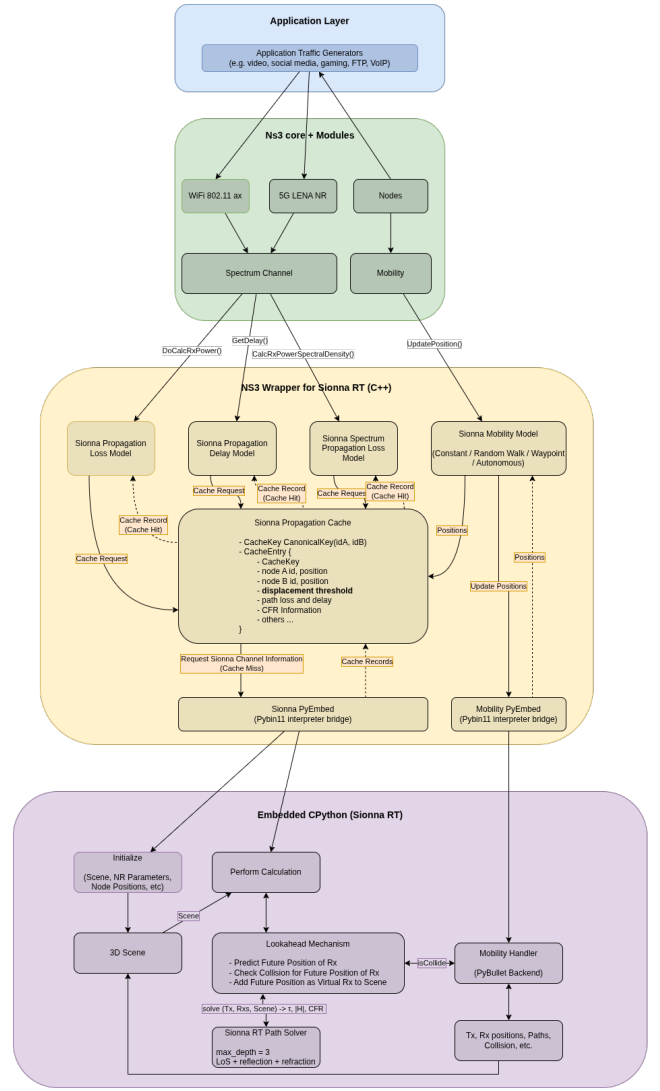


Figure 1: Architecture of the integrated simulation framework.

computed parameters. Algorithm 1 formalizes this lookup procedure.

To further reduce cache misses, the framework proactively pre-populates the cache with channel parameters for predicted future node positions. Given each node's current position p_i and velocity v_i , a set of future positions is computed over a discrete set of look-ahead intervals $\{\Delta t_k\}$. These predicted positions are inserted into the Sionna RT scene as *virtual receivers* alongside the real receivers. A single batched ray tracing call then computes paths for all positions simultaneously, and the resulting parameters are stored in the cache. Algorithm 2 describes this pre-computation step.

However, ns-3 cannot directly call the Sionna RT engine, as the two components are written in different programming languages. A wrapper class is therefore introduced to bridge ns-3 and the Sionna RT engine.

Algorithm 1 Propagation Query with Coherence Cache

Require: transmitter position p_{tx} , receiver position p_{rx} , coherence distance d_{coh}

Ensure: channel parameters $\mathbf{H}$

```

1:  $key \leftarrow \text{hash}(p_{tx}, p_{rx})$ 
2: if  $key \in \text{cache}$  then
3:    $(\mathbf{H}_c, p_{tx}^s, p_{rx}^s) \leftarrow \text{cache}[key]$ 
4:    $d \leftarrow \|p_{tx} - p_{tx}^s\| + \|p_{rx} - p_{rx}^s\|$ 
5:   if  $d < d_{coh}$  then
6:     return  $\mathbf{H}_c$                                 {cache hit}
7:   end if
8: end if
9:  $\mathbf{H} \leftarrow \text{SionnaRT.ComputePaths}(p_{tx}, p_{rx})$     {cache miss}
10:  $\text{cache}[key] \leftarrow (\mathbf{H}, p_{tx}, p_{rx})$ 
11: return  $\mathbf{H}$ 
    
```

Algorithm 2 Adaptive Virtual Receiver Generation

Require: node positions $\{p_i\}_{i=1}^N$, velocities $\{v_i\}_{i=1}^N$, look-ahead intervals $\{\Delta t_k\}_{k=1}^K$, transmitter position p_{tx}

Ensure: cache pre-populated for predicted future positions

```

1:  $P_{\text{virtual}} \leftarrow \emptyset$ 
2: for each node  $i \in \{1, \dots, N\}$  do
3:   for each interval  $\Delta t_k \in \{\Delta t_k\}$  do
4:      $p_{\text{future}} \leftarrow p_i + v_i \cdot \Delta t_k$ 
5:      $P_{\text{virtual}} \leftarrow P_{\text{virtual}} \cup \{p_{\text{future}}\}$ 
6:   end for
7: end for
8:  $P_{\text{all}} \leftarrow \{p_i\} \cup P_{\text{virtual}}$     {real + virtual receivers}
9:  $\mathbf{H}_{\text{all}} \leftarrow \text{SionnaRT.ComputePaths}(p_{tx}, P_{\text{all}})$  {single batched call}
10: for each  $p_j \in P_{\text{all}}$  do
11:    $\text{cache}[\text{hash}(p_{tx}, p_j)] \leftarrow (\mathbf{H}_{\text{all}}[j], p_{tx}, p_j)$ 
12: end for
    
```

The bottom layer hosts the Sionna RT engine, which performs the physical ray tracing computation. Before simulation begins, the engine is initialized with radio propagation parameters, antenna configuration, and the 3D scene file name passed from the ns-3 side. During simulation, the path solver computes path loss, time of arrival, angle of arrival, and angle of departure for each requested receiver position by tracing rays through the 3D environment [12]. These computations execute on the GPU, and the results are returned to the cache layer via the wrapper class.

All experiments are conducted on an Asus Ascent GX10 AI supercomputer equipped with an NVIDIA GB10 Grace Blackwell Superchip: a 20-core Arm-based NVIDIA Grace CPU paired with an NVIDIA Blackwell GPU, 128 GB of unified LPDDR5x memory, and NVIDIA ConnectX-7 networking.

4.1. Sionna Propagation Cache Mechanism

4.1.1. Displacement-Based Cache Validity

The cache exploits spatial coherence: when a node moves only a small distance from its last traced position, channel parameters change negligibly. Cache validity is therefore determined by spatial displacement rather than elapsed time.

The cache stores a record for each transmitter–receiver pair in the ns-3 simulation. Each cache entry is indexed by a canonical unordered node pair,

$$\kappa(a, b) = (\min(I_a, I_b), \max(I_a, I_b)), \quad (1)$$

where I_a and I_b are ns-3 node identifiers. Each entry stores the node positions $\mathbf{x}_a^0$ and $\mathbf{x}_b^0$, path loss, transmission delay, channel frequency response (CFR), and line-of-sight (LoS) state.

The distance between Tx and Rx is calculated using

$$d_{a,b}^0 = \|\mathbf{x}_a^0 - \mathbf{x}_b^0\|_2, \quad (2)$$

and the validity radius is given by

$$\Delta_{a,b} = \max \left(\Delta_{\min}, \alpha \frac{0.886 d_{a,b}^0}{N_{\text{tx,col}}} \right). \quad (3)$$

where α is set to 0.4 and $\Delta_{\min}$ is set to 0.1 m as default values.

On each propagation query for a Tx–Rx pair, the current positions $\mathbf{x}_a(t)$ and $\mathbf{x}_b(t)$ are compared against the positions stored at trace time. A valid cache entry exists if

$$\|\mathbf{x}_a(t) - \mathbf{x}_a^0\|_2 < \Delta_{a,b} \quad \text{and} \quad \|\mathbf{x}_b(t) - \mathbf{x}_b^0\|_2 < \Delta_{a,b}. \quad (4)$$

If both inequalities are satisfied, the cached parameters are returned directly. Otherwise, the lookup is a cache miss and the snapshot is refreshed.

4.1.2. Snapshot Refresh

When a cache miss occurs, the framework refreshes all Tx–Rx pairs rather than only the affected link. First, the positions of every node that uses `SionnaMobilityModel` are synchronized into the Python Sionna scene. A single batched ray tracing call is then issued for all Tx–Rx pairs by invoking `perform_calculation` in `sionnart.py`. When the computation completes, the returned records are deserialized via `pybind11` and inserted as new cache entries. For each record, the cache stores

$$(\tau_{a,b}, L_{a,b}, \mathbf{h}_{a,b}, \mathbf{H}_{a,b}, \mathbf{f}, \text{LoS}_{a,b}, \mathbf{x}_a^0, \mathbf{x}_b^0, \Delta_{a,b}), \quad (5)$$

where $\tau_{a,b}$ is delay, $L_{a,b}$ is path loss, $\mathbf{h}_{a,b}$ is the scalar CFR, $\mathbf{H}_{a,b}$ is the MIMO CFR, and $\mathbf{f}$ is the frequency vector. The old cache records are replaced only if the new results contain usable entries.

4.1.3. Lookup and Fallback Behavior

ns-3 queries propagation information for each Tx–Rx pair. For a normal link, the cache searches the canonical key, tests displacement validity, and returns the first valid entry found. If no valid entry exists, the cache refreshes all records and retries the same lookup sequence.

In Sionna RT, links between nodes with the same radio role, such as Tx–Tx or Rx–Rx, cannot be computed. In such cases, a dummy entry with large path loss and delay is returned instead. If Sionna RT does not return usable data for the requested link after refresh, the cache falls back to the stochastic Friis model and constant-speed delay.

Before calling Sionna RT, the cache checks the estimated received power using the Friis model to avoid unnecessary ray tracing. If

$$P_{\text{Friis}}^{\text{rx}} + M < P_{\text{weak}}, \quad (6)$$

where $P_{\text{weak}} = -95$ dBm and $M = 6$ dB by default, the link is treated as too weak to justify ray tracing. In that case, Friis loss and constant-speed delay are used directly.

4.1.4. Derived MIMO Caches

The records returned by the Sionna RT engine store the normalized element-level MIMO CFR. However, ns-3 requires derived quantities such as port-level channels, PSD-scaled matrices, and effective post-precoding gains. These quantities depend not only on geometry but also on antenna configuration, beamforming vectors, PSDs, and precoders. They are computed from the element-level CFR together with ns-3-side parameters and stored as derived cache entries.

Computation proceeds as follows. For a given beam configuration, the element-level channel is projected onto antenna ports. For resource block q , transmit port p_t , and receive port p_r , the port-level channel has the form

$$G_{p_r, p_t}[q] = \sum_{i \in \mathcal{E}_{p_r}} \sum_{j \in \mathcal{E}_{p_t}} w_{r,i} w_{t,j} \tilde{H}_{i,j}[q], \quad (7)$$

where $\mathcal{E}_{p_r}$ and $\mathcal{E}_{p_t}$ are the receive and transmit element sets associated with the ports, and $w_{r,i}$ and $w_{t,j}$ are the beamforming weights. This port CFR is cached using antenna identifiers, beamforming-vector hashes, port counts, and link direction.

For spectrum propagation, the port CFR is scaled by the input PSD:

$$C_{p_r, p_t}[q] = \sqrt{S[q]} G_{p_r, p_t}[q], \quad (8)$$

where $S[q]$ is the PSD value on resource block q . The most recent PSD-scaled matrix is cached using a PSD hash and PSD size.

When a precoding matrix $\mathbf{W}[q]$ is provided, the effective gain per resource block is

$$g[q] = \sum_{p_r} \sum_s \left| \sum_{p_t} G_{p_r, p_t}[q] W_{p_t, s}[q] \right|^2. \quad (9)$$

These gains are cached using the port-CFR key, precoder hash, precoder dimensions, and output resource-block count. This avoids repeated beamforming and precoding reductions while the underlying propagation entry remains valid.

4.2. Adaptive Virtual Receiver Generation

The coherence cache described in Section 4.1 is reactive: a new ray tracing call is issued only after a cache entry becomes invalid. Adaptive virtual receiver generation is a complementary proactive mechanism that reduces the number of such cache misses. During each cache refresh, a small number of temporary receiver positions are placed ahead of each UE along its trajectory. These positions are not new network nodes; they are spatial samples for the same UE at predicted future locations. Including them in the same Sionna RT call returns both present and near-future channel samples at no additional ray tracing overhead.

Let the UE position and speed be

$$\mathbf{x}_r(t) = [x_r(t), y_r(t), z_r(t)]^T, \quad v_r. \quad (10)$$

Prediction of virtual receiver positions is not performed for a stationary receiver (i.e., $v_r \leq 10^{-6}$ m/s). For a moving UE, the distance to the nearest transmitter is

$$d_r(t) = \min_{b \in \mathcal{B}} \|\mathbf{x}_r(t) - \mathbf{x}_b\|_2. \quad (11)$$

The valid displacement Δ_r is computed using the coherence formula from Section 4.1.1, based on $d_r(t)$ and a default $\Delta_{\min} = 0.5$ m. The current sample is predicted to expire after

$$T_{\text{expire}, r} = \frac{\Delta_r}{v_r}. \quad (12)$$

This is the time for the UE to travel the full valid displacement at its current speed. Virtual receivers are generated only if $T_{\text{expire}, r} < T_{\min}$, with $T_{\min} = 1$ s.

The generation of virtual receivers is closely tied to the underlying mobility model of the UE. For UEs with predictable trajectories, such as those using a Waypoint mobility model, the direction of motion is stable and virtual receivers are reliably placed along the projected path. In contrast, for unpredictable movement patterns, such as a random walk, predicting future positions is unreliable.

To differentiate these scenarios, the framework assesses trajectory stability by checking that the dot product of the current and previous unit direction vectors satisfies

$$\mathbf{u}_r(t - \Delta t)^T \mathbf{u}_r(t) \geq \rho_{\min}, \quad (13)$$

where $\rho_{\min} = 0.7$, and the prediction direction is defined by the normalized horizontal displacement,

$$\mathbf{u}_r(t) = \frac{[\Delta x_r(t), \Delta y_r(t), 0]^T}{\sqrt{\Delta x_r(t)^2 + \Delta y_r(t)^2}}. \quad (14)$$

If this stability condition is not met, no virtual receivers are generated. This prevents wasting computational resources on ray tracing for locations the UE is unlikely to visit.

For stable trajectories, the inter-sample spacing s_r and prediction horizon D_r are

$$s_r = \beta \Delta_r, \quad D_r = v_r T_h, \quad (15)$$

with $\beta = 1.1$ and $T_h = 3$ s. The number of virtual receivers is

$$K_r = \min \left(K_{\max}, \left\lceil \frac{D_r}{s_r} \right\rceil \right), \quad (16)$$

where $K_{\max} = 3$. Their positions are calculated as

$$\mathbf{x}_{r,k}^{\text{virt}}(t) = \mathbf{x}_r(t) + k s_r \mathbf{u}_r(t), \quad k = 1, \dots, K_r. \quad (17)$$

Thus, only positions along the UE trajectory are sampled, and the receiver height remains unchanged.

4.3. Propagation Calculation

On a cache miss, ns-3 synchronizes the positions of all UEs to the Python scene and adds eligible virtual receivers before issuing a single batched ray tracing call. Let $\mathcal{R}$ be the physical receiver set and $\mathcal{V}_r$ the virtual receivers generated for UE r ; ray tracing is performed over the augmented set

$$\mathcal{R}^+ = \mathcal{R} \cup \bigcup_{r \in \mathcal{R}} \mathcal{V}_r. \quad (18)$$

The Sionna `PathSolver` is called with maximum depth 3, line-of-sight paths, specular reflections, refraction, and synthetic arrays enabled; diffuse reflection is disabled [12, 2, 10]. The result is a single geometrically consistent snapshot of propagation parameters for all present and predicted receiver positions.

For each Tx-Rx pair (b, r) , the solver returns a set of propagation paths $\{(\tau_{b,r,p}, \mathbf{h}_{b,r,p})\}$ indexed by p . The reported propagation delay is the minimum non-negative path delay,

$$d_{b,r} = \text{round} \left(10^9 \min_{p: \tau_{b,r,p} \geq 0} \tau_{b,r,p} \right) [\text{ns}]. \quad (19)$$

Combining all paths across the OFDM bandwidth yields the MIMO channel frequency response

$$\mathbf{H}_{b,r} \in \mathbb{C}^{N_{\text{rx}} \times N_{\text{tx}} \times N_f}, \quad (20)$$

where N_{rx} , N_{tx} , and N_f are the numbers of receive antenna elements, transmit antenna elements, and resource blocks, respectively.

The average channel power and path loss are then computed as

$$P_{b,r} = \frac{1}{N_{\text{rx}} N_{\text{tx}} N_f} \sum_{i,j,q} |H_{b,r}[i,j,q]|^2, \quad (21)$$

$$L_{b,r} = -10 \log_{10}(P_{b,r}). \quad (22)$$

If $P_{b,r} = 0$, the implementation exports $L_{b,r} = 200$ dB to indicate no signal. The channel is then normalized as

$$\tilde{H}_{b,r}[i,j,q] = \frac{H_{b,r}[i,j,q]}{\sqrt{P_{b,r}}}, \quad (23)$$

separating large-scale attenuation from small-scale spatial and frequency selectivity: $L_{b,r}$ is consumed by the ns-3 `PropagationLossModel`, while $\tilde{H}_{b,r}$ is passed to the spectrum channel matrix. For ns-3 export, the tensor is serialized according to the `MatrixArray` page-major, column-major layout: the flat index is

$$k = q N_{\text{tx}} N_{\text{rx}} + j N_{\text{rx}} + i, \quad (24)$$

where q , j , and i index resource block, transmit antenna element, and receive antenna element, respectively.

After the records are returned to ns-3, the temporary virtual receivers are removed from the Python scene. The ns-3 propagation cache stores all returned records for both real and virtual receiver positions using the same displacement-validity rule, so that future queries for nearby positions are served from the cache without invoking the ray tracer again.

4.4. Energy Model for 5G NR gNB and UEs

To evaluate the network energy consumption during simulations, the ns-3 energy framework is extended by implementing custom `NrGnbEnergyModel` and `NrUeEnergyModel` modules [21]. These models track the operational states of the NR spectrum PHY layer, represented by the state space $\mathcal{S} = \{\text{IDLE}, \text{RX_CTRL}, \text{RX_DATA}, \text{TX}, \text{CCA_BUSY}\}$. The power draw of each node is computed from the time spent in each state.

For the User Equipment (UE), the total energy consumed is computed from its base power profile:

$$E_{\text{UE}} = \int_0^T P_{\text{UE}}(s(t)) dt = \sum_{s \in \mathcal{S}} P_s^{\text{UE}} \cdot \tau_s, \quad (25)$$

where P_s^{UE} is the power consumption in state s , and τ_s is the cumulative time spent in that state.

By contrast, the Next Generation NodeB (gNB) power model accounts for both a baseline operating load and the scaling effects of its antenna array:

$$E_{\text{gNB}} = \int_0^T P_{\text{gNB}}(s(t)) dt = \sum_{s \in \mathcal{S}} (P_{\text{fixed}} + N_{\text{ant}} P_s^{\text{gNB}}) \tau_s,$$

(26)

where P_{fixed} is the constant hardware power draw, N_{ant} is the number of antennas, and P_s^{gNB} is the per-antenna state power.

The time a node spends in active states (τ_s) depends on the network's link adaptation, which relies on the Signal-to-Interference-Plus-Noise Ratio (SINR). For a receiver i on subcarrier k , the SINR is calculated as:

$$\text{SINR}_{i,k} = \frac{P_{\text{tx}} G_{\text{tx}} G_{\text{rx}} |H_{i,k}|^2}{N_0 W_k + \sum_{j \neq i} P_{\text{tx}} G_{\text{tx}} G_{\text{rx}} |H_{j,k}|^2}, \quad (27)$$

where H is the channel coefficient supplied by the propagation model described in Section 4.3.

The gNB uses this SINR to select a Modulation and Coding Scheme (MCS); a higher SINR enables a more spectrally efficient MCS, which reduces the time spent in the power-intensive transmit state ($\tau_{\text{TX},i}$). Overall energy efficiency, measured as energy per delivered bit, is then:

$$\mathcal{E}_b = \frac{E_{\text{node}}}{\sum_i \eta_i W_i \tau_{\text{TX},i}}, \quad (28)$$

where E_{node} is the total energy of the UE or gNB.

This formulation shows that energy efficiency $\mathcal{E}_b$ is highly sensitive to the channel coefficient H . A simple analytical model such as Friis provides a coarse approximation, whereas ray tracing yields an environment-specific, geometry-accurate channel. Section 6 examines how this choice of propagation model affects the resulting energy metrics.

5. Simulation Framework Benchmarking

This section benchmarks the computational viability of our integrated simulation framework. Specifically, we evaluate the cost of replacing the standard stochastic ns-3 propagation model with the deterministic Sionna RT engine embedded directly within the simulator process. To establish a baseline for our framework's efficiency, we compare three distinct channel backends: **pure ns-3**, **ns3sionna**, and our embedded **sionnart** module. The **pure ns-3** backend utilizes the conventional ns-3 stochastic propagation model, while the **ns3sionna** [13] implementation relies on communicating with an external Python-based Sionna RT server via ZMQ sockets. Finally, **sionnart** runs Sionna RT directly inside the simulator's address space using pybind11 [11, 12, 14]. To benchmark the computational cost of the three channel backends, the same network topology — a single IEEE 802.11ax access point and $N \in \{1, 2, 4, 8, 16, 32\}$ stations — and the same application traffic profile are used. The benchmarking is done on IEEE 802.11ax instead of 5G NR as the **ns3sionna** [13] is tested only on IEEE 802.11ax and not on 5G NR. Keeping topology and traffic fixed ensures that

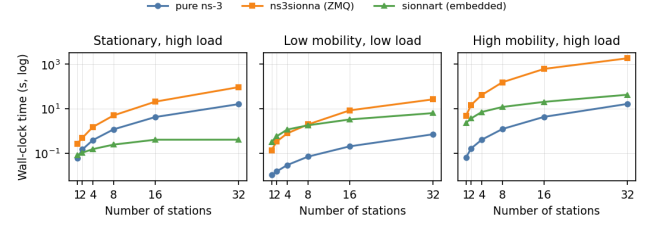


Figure 2: Total simulation runtime of the three channel backends as the number of stations increases. The communication with external Sionna RT server via ZMQ in **ns3sionna** causes large overhead. The embedded **sionnart** backend avoids the round trip overhead, while the pure ns-3 baseline provides the analytical-channel lower bound.

Table 1

Total simulation runtime in seconds for representative station counts.

Regime	Backend	$N = 1$	$N = 4$	$N = 16$	$N = 32$
Stationary	pure ns-3	0.061	0.385	4.230	16.160
Stationary	ns3sionna	0.266	1.504	20.826	93.834
Stationary	sionnart	0.082	0.152	0.403	0.404
Low mobility	pure ns-3	0.010	0.029	0.203	0.708
Low mobility	ns3sionna	0.137	0.778	8.453	26.439
Low mobility	sionnart	0.320	1.153	3.292	6.370
High mobility	pure ns-3	0.064	0.400	4.320	16.516
High mobility	ns3sionna	4.821	41.312	622.365	1871.600
High mobility	sionnart	2.293	7.124	20.405	42.772

runtime differences are caused solely by the channel backend. With the same setup, three different scenarios are simulated:

- **stationary_high_load**: static nodes and high packet rate;
- **low_mob_low_load**: slow random-walk mobility and low packet rate;
- **high_mob_high_load**: fast random-walk mobility and high packet rate.

The last scenario is to stress test the ray tracing because mobility repeatedly invalidates the cache of channels while the high packet rate keeps the simulator issuing propagation queries.

5.1. Total Simulation Time Scaling

Figure 2 shows the total time taken to finish the simulation over the number of stations N with different channel backends. The y-axis is in logarithmic scale because the total simulation time spans more than four orders of magnitude between the smallest and largest runs. The same results are presented in Table 1.

Scaling behavior differs across scenarios. In the stationary scenario, the apparent advantage of **sionnart** over the **pure ns-3** baseline is a consequence of the cache (Section 4.1.1), not evidence that ray tracing

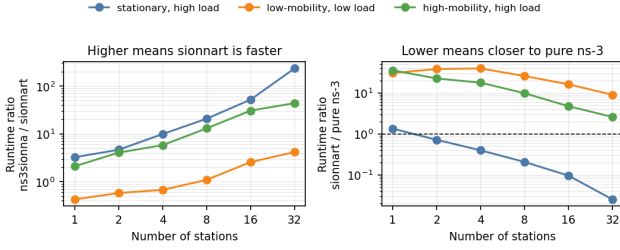


Figure 3: The left graph shows the ratio of **ns3sionna** runtime to **sionnart** runtime. The right graph is the ratio of **sionnart** runtime to **pure ns-3** baseline runtime.

is intrinsically faster than Friis evaluation. Because no UE moves, the ray-tracer is invoked only once at $t = 0$; all subsequent propagation queries are served from the cached channel snapshot while the analytical baseline still re-evaluates its loss model at every packet arrival. This cache-driven effect produces an approximately $40\times$ wall-clock reduction at $N = 32$, and would collapse to significant overhead if mobility were introduced without the cache. Even in this stationary case, the **ns3sionna** backend is slower than both **pure ns-3** and **sionnart** due to the round-trip overhead of communicating with the external Sionna RT server.

In the low-mobility, low-load scenario, the results show the opposite trade-off. When the station count N is small, **sionnart** is slower than **ns3sionna** because there are few propagation queries and the cost of each query is dominated by the fixed overhead of the embedded Python runtime and Sionna scene initialization. As N grows, batching and cache reuse become more effective; by $N = 32$, **sionnart** is $4.15\times$ faster than **ns3sionna**. It remains $9\times$ slower than the **pure ns-3** baseline because the traffic load is low enough that the stochastic propagation model remains computationally cheap.

In the high-mobility, high-load scenario, **ns3sionna** scales poorly with station count because each cache miss requires copying data between processes, waiting for a round-trip between the two servers, and updating the Sionna side state. The **sionnart** backend still must invoke the ray-tracer on each cache miss, but it avoids inter-process communication overhead and keeps the cache inside the same process. As a result, the gap between **sionnart** and **ns3sionna** grows monotonically from $2.1\times$ at $N = 1$ to $43.8\times$ at $N = 32$.

5.2. Speedup Analysis

Figure 3 shows the runtime ratios. The left panel shows $T_{ns3sionna}/T_{sionnart}$ and a larger value means that **sionnart** is faster than **ns3sionna**. The right panel shows $T_{sionnart}/T_{pure\ ns-3}$ and a value below one means that **sionnart** is faster than **pure ns-3** baseline. The same results are shown in Table 2.

Table 2

Ratios at $N = 32$. The first ratio is $T_{ns3sionna}/T_{sionnart}$; the second is $T_{sionnart}/T_{pure\ ns-3}$.

Scenario	Speedup over ns3sionna	Cost relative to pure ns-3
Stationary, high load	$232.3\times$	$0.025\times$
Low mobility, low load	$4.15\times$	$9.00\times$
High mobility, high load	$43.8\times$	$2.59\times$

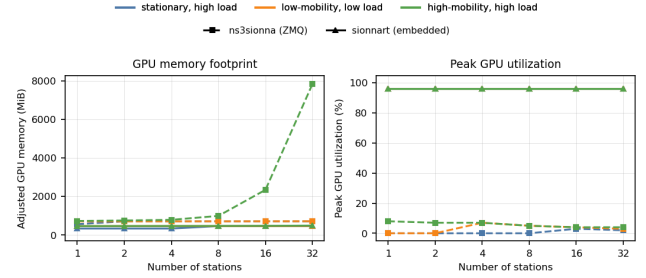


Figure 4: Reported GPU memory footprint and peak utilization for the two Sionna-backed backends.

According to these ratios, embedding Sionna RT using pybind11 helps to improve the performance of simulation. In two server design, a cache miss must send data from ns-3 to a separate Python process and wait for the reply. In the embedded design, ns-3, the cache, Sionna RT, all run inside the same process. This removes much of the communication cost and the remaining cost is the actual ray tracing, which is still needed when the experiment uses site-specific propagation.

The two-server design also has its own merits: a separate Sionna RT server can run on a different machine and multiple ray tracing engines can run in parallel. For the use case considered here, however, co-locating the ray tracing engine with ns-3 is sufficient.

5.3. GPU Footprint and Utilization

Figure 4 shows the GPU memory footprint and processing core utilization for each scenario. For **sionnart**, allocated memory is almost flat. It is 337 MiB for the smallest stationary scenario, about 465-469 MiB for most scenarios, and 491 MiB for the largest high-mobility scenario. This is because most of the memory usage is due to the Sionna RT scene and increasing number of stations in the same scene, not the number of packets processed by ns-3. By contrast, **ns3sionna** grows significantly in the high-mobility scenario. The memory increases from 730 MiB at $N = 1$ to 7858 MiB at $N = 32$.

The peak-utilization panel reports the maximum observed value, not the average load. **sionnart** reaches up to 96% peak utilization during batched channel calculations, but these bursts are short. By contrast,

`ns3sionna` has lower peak utilization but occupies the GPU more continuously.

In summary, `sionnart` maintains a nearly flat GPU memory footprint because the scene geometry and per-link state are reused across queries. `ns3sionna` requires growing memory because it must retain per-link ray tracing state for each additional mobile link.

6. Application Case Study: massive MIMO 5G Evaluation

Having successfully verified the computational efficiency and scalability of our embedded simulation framework, we now demonstrate its practical utility through a realistic 3GPP NR deployment case study [9, 1]. This section showcases how the integrated Sionna RT backend allows researchers to evaluate end-to-end 5G massive MIMO networks. We assess critical application-level KPIs (throughput, packet loss, delay, jitter), propagation-level KPIs (path loss, path delay, CQI, MCS), and overall energy consumption. To provide a comprehensive evaluation, we simulate three distinct radio environments—free space, urban macro, and urban micro—using gNB antenna array sizes ranging from 2×2 to 8×8 , under varying traffic loads.

6.1. Experiment Configuration

In each simulation run, one gNB and 20 UEs are used. The UE count is fixed to 20 to keep per-step GPU memory and simulation wall-clock time within practical bounds on the evaluation hardware; scaling to denser deployments is outside the scope of this work. The gNB antenna array is varied over 2×2 , 4×4 , and 8×8 ; each UE uses a fixed 2×2 array. The simulations are tested in three different radio environments:

- **free space:** 2 GHz carrier, 35 m gNB height, and UE distances from 50 m to 1000 m;
- **urban macro:** 3.5 GHz carrier, 25 m gNB height, and UE distances from 35 m to 500 m;
- **urban micro:** 28 GHz carrier, 10 m gNB height, and UE distances from 10 m to 200 m.

The carrier frequencies, antenna heights, and UE distance ranges are aligned with the 3GPP TR 38.901 reference deployment configurations [1]. The 2 GHz carrier for the free-space baseline corresponds to the FR1 sub-3 GHz band; the 3.5 GHz urban macro carrier matches the primary 5G NR band (n78, FR1); and the 28 GHz urban micro carrier represents the FR2 mmWave band (n257/n261). The gNB heights of 25 m (urban macro) and 10 m (urban micro) follow the TR 38.901 UMa and UMi reference values respectively, while the 35 m free-space height is consistent with rural macro conventions. The UE distance ranges are chosen to cover the minimum coupling distance and approximate cell radius

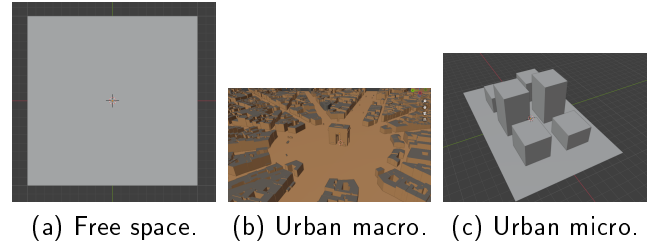


Figure 5: The three simulated radio environments.

defined for each scenario in TR 38.901 Table 7.2-1. The single-cell, 20-UE configuration is consistent with the single-sector system-level evaluation methodology of TR 38.901 Table A.1. The free-space scenario is the baseline, the urban macro scenario represents a sub-6 GHz deployment, and the urban micro scenario represents the most demanding physical setting as it combines a mmWave carrier with geometry-sensitive, short-range links. The 3D visualizations of these three environments are shown in Figure 5.

For scheduling UEs, a TDMA proportional fair scheduler, and full-buffer TDD slots are used. On top of the realistic wireless channel model, different applications with different traffic loads are tested. The traffic profile includes video, voice over IP (VoIP), social media, gaming, and FTP. The per-application UE distribution — 12 (60%), 2 (10%), 2 (10%), 1 (5%), and 3 (15%) for video, VoIP, social media, gaming, and FTP respectively — follows the traffic mix proportions reported in [22], and is applied identically for both DL and UL. The data rate for each application according to load levels are summarized in Table 3.

6.2. Free-Space Scenario

The free-space scenario serves as the baseline because it has no building blockage; distance is the sole large-scale factor. UEs are distributed up to 1000 m from the gNB, so the scenario combines strong near-field links with very weak cell-edge links.

6.2.1. Application-related Metrics

Figure 6 shows per-application throughput, packet loss, delay, and jitter for both directions and all antenna configurations. The stochastic ns-3 model outperforms Sionna RT in average downlink throughput: at 8×8 , ns-3 delivers 4.85 Mbit s^{-1} with 21.6% loss versus 3.77 Mbit s^{-1} with 26.6% loss for Sionna RT, with the same ordering at 2×2 . The gap reflects the large cell radius: UEs at the cell edge experience greater path loss under ray tracing than the stochastic model predicts. Array scaling yields only marginal throughput gain in both simulators because the bottleneck is cell-edge path loss, not beamforming resolution.

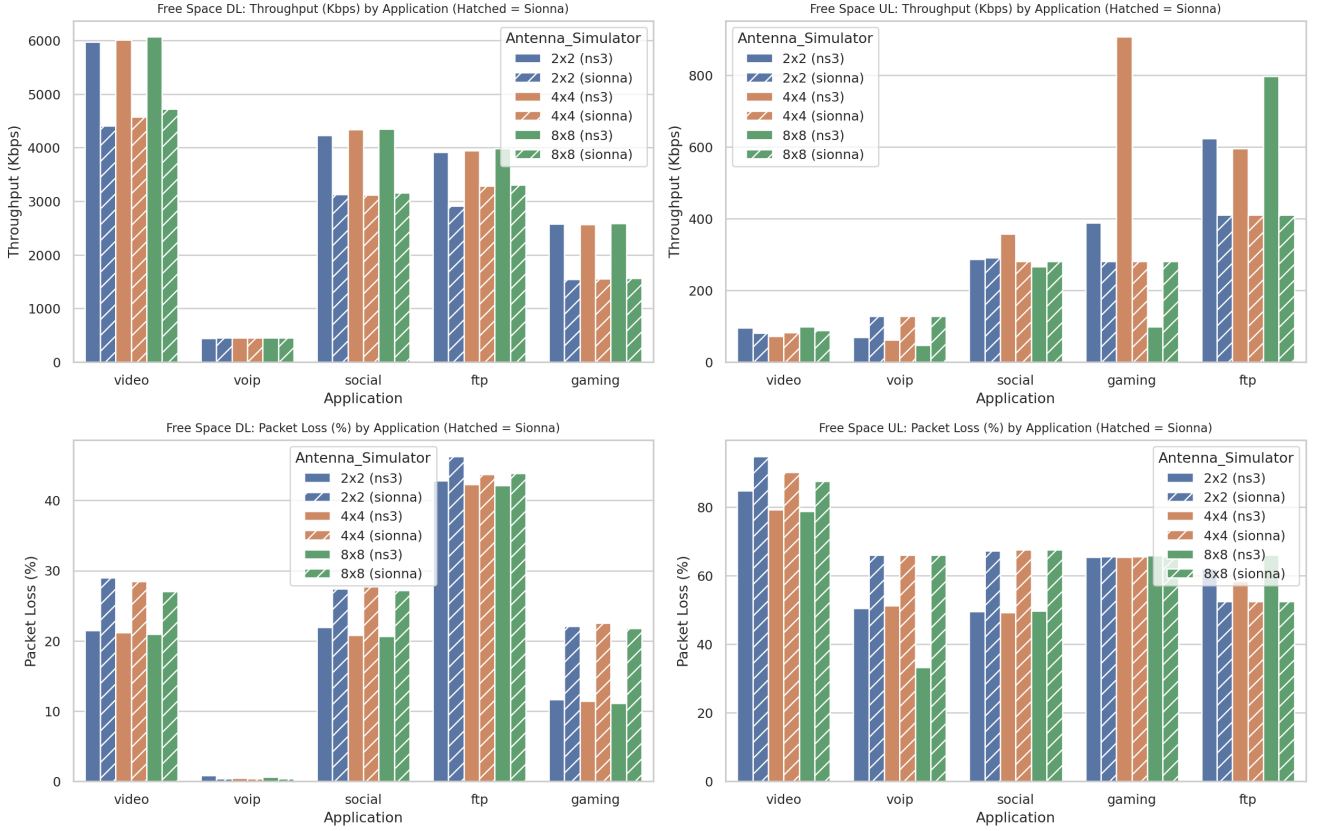
6.2.2. Propagation-related Metrics

Average path loss is 94.2 dB and estimated receive power is -48.2 dBm for both channel models (Fig. 9).

Table 3

Application data rate by application and load.

Application	Flows	Low		Medium		High	
		DL (Mbps)	UL (Mbps)	DL (Mbps)	UL (Mbps)	DL (Mbps)	UL (Mbps)
Video	12 DL / 12 UL	24.000	0.300	96.000	1.200	300.000	3.750
VoIP	2 DL / 2 UL	0.128	0.032	0.256	0.064	2.000	0.500
Social	2 DL / 2 UL	1.000	0.050	10.000	0.500	40.000	2.000
Gaming	1 DL / 1 UL	0.100	0.025	1.000	0.250	10.000	2.500
FTP	3 DL / 3 UL	3.000	0.150	30.000	1.500	150.000	7.500
Total	20 DL / 20 UL	28.228	0.557	137.256	3.514	502.000	16.250


Figure 6: Free-space application-level throughput and packet loss by application, direction, and gNB array size. Solid bars: ns-3; hatched bars: Sionna RT.

The ns-3 model reports a mean path delay of 3059 ns while Sionna RT reports 2237 ns; both are consistent with radio path lengths of hundreds of metres. Channel quality improves monotonically with array size in both simulators, with ns-3 reporting marginally higher values than Sionna RT at each configuration. The difference is small in free space because most links are near-line-of-sight; beamforming gain raises link quality but cannot overcome path loss for the most remote UEs.

6.2.3. Energy Consumption-related Metrics

gNB power consumption increases with array size in both simulators (Fig. 10). In ns-3, the gNB consumes 31.16 W (2×2), 64.58 W (4×4), and 199.39 W

(8×8); Sionna RT reports 30.08 W, 59.08 W, and 173.50 W respectively. UE power is similar across all configurations because the UE array is fixed at 2×2 . The throughput-per-watt trade-off is poor for larger arrays in this scenario: 8×8 consumes roughly six times the gNB power of 2×2 while delivering only marginal throughput gains. Array gain cannot offset the scheduling pressure of serving many long-distance UEs under high traffic load.

6.3. Urban Macro Scenario

The urban macro scenario is based on the Charles de Gaulle Étoile in Paris, a dense urban environment with tall buildings and complex obstruction patterns. The gNB is mounted at 25 m (3GPP UMa reference height

[1]) near the Arc de Triomphe, and the 3.5 GHz carrier (FR1 n78) is used. The combination of high building density, sub-6 GHz carrier, and macro-cell distances makes this a representative and challenging 5G NR deployment scenario.

6.3.1. Application-related Metrics

Figure 7 shows per-application metrics for the urban macro scenario. Among data-intensive applications, ns-3 outperforms Sionna RT at 2×2 for FTP (16.8 Mbit s⁻¹ vs. 15.2 Mbit s⁻¹), but Sionna RT surpasses ns-3 at 4×4 and 8×8 (19.5 Mbit s⁻¹ vs. 18.5 Mbit s⁻¹). Sionna RT benefits more from larger arrays because ray tracing resolves the multipath structure — reflections and diffractions from surrounding buildings — that enables spatial multiplexing; the stochastic model cannot leverage this geometric structure. For video streaming, ns-3 consistently achieves higher throughput than Sionna RT across all antenna configurations.

Downlink packet loss decreases steadily with array size in both simulators. Sionna RT shows higher video packet loss (9.5%, 7.6%, 4.6% at 2×2 , 4×4 , and 8×8 , respectively); at 2×2 , ns-3 performs better for video but worse for FTP. Uplink packet loss is severe in both simulators (15% for VoIP up to near-total loss for FTP at 2×2), reflecting the limited UE transmit power against urban blockage.

6.3.2. Propagation-related Metrics

Average path loss is 83.5 dB and received power is -37.5 dBm, nearly identical between the two channel models across all array sizes (Fig. 9). The simulators diverge on path delay: ns-3 reports ≈ 220 ns while Sionna RT reports ≈ 360 ns. The elevated delay in Sionna RT is attributable to its multipath-aware propagation model: ray-traced paths include reflected and diffracted components whose longer geometric lengths contribute to the reported delay, whereas the stochastic model reports only the direct-path delay. This mechanistic difference persists in the urban micro scenario as well (Section 6.4).

Channel quality is consistently lower in Sionna RT than in ns-3 across all array configurations, and improves with array size in both simulators. Consistent with the free-space baseline (Section 6.2), scaling from 2×2 to 8×8 confirms that larger arrays overcome urban obstruction through beamforming gain.

6.3.3. Energy Consumption-related Metrics

gNB power scales with both array size and traffic load in both simulators (Fig. 10). Under low traffic, both report approximately 25 W, 40 W, and 100 W for 2×2 , 4×4 , and 8×8 . Sionna RT reports slightly higher power than ns-3 under high load because its lower predicted channel quality leads to different resource-block allocations and transmission durations. UE power consumption increases from 0.058 W at low load to

0.095 W at high load and is nearly identical between the two simulators, as UEs are constrained to a 2×2 array throughout.

6.4. Urban Micro Scenario

The urban micro scenario simulates a dense street-level environment where the gNB is deployed at 10 m below the surrounding rooftops. The combination of a 28 GHz mmWave carrier, short inter-site distances, and frequent NLOS conditions makes this the most demanding propagation setting among the three scenarios.

6.4.1. Application-related Metrics

Figure 8 shows per-application metrics for the urban micro scenario. Sionna RT consistently achieves higher downlink throughput than ns-3 for data-intensive applications. FTP throughput reaches nearly 20 Mbit s⁻¹ at 4×4 and 8×8 under Sionna RT, compared to ≈ 15 Mbit s⁻¹ for ns-3. This advantage reflects the multipath components (reflections, diffractions) resolved by the ray tracer, which enable spatial multiplexing gains that the stochastic model cannot capture — consistent with the trend observed in the macro scenario (Section 6.3).

Downlink packet loss under ns-3 reaches nearly 30% for FTP and gaming at 2×2 , while Sionna RT shows lower loss for these applications but higher loss for video at 2×2 . Scaling to 8×8 reduces downlink packet loss to near-zero for most applications in both simulators, demonstrating that massive MIMO can overcome urban blockage. Uplink remains severely loss-limited (60%–90% in both simulators), reflecting the constraint that UE transmit power is insufficient to overcome dense urban obstacles without receive beamforming at the gNB.

6.4.2. Propagation-related Metrics

Average path loss is 95.5 dB and received power is -62.5 dBm, consistent across all antenna sizes and both simulators (Fig. 9). As in the macro scenario (Section 6.3), Sionna RT reports a higher mean path delay (178 ns vs. 120 ns for ns-3), consistent with its multipath-aware propagation accounting over the dense street-level geometry.

Sionna RT also predicts lower channel quality than ns-3, with a more pronounced gap at 2×2 that reflects the harsher fading and interference resolved by the ray tracer. Both simulators show significant channel quality improvement as the array scales to 8×8 , further confirming that larger arrays are essential for meeting latency and reliability requirements in dense urban mmWave deployments.

6.4.3. Energy Consumption-related Metrics

The gNB energy scaling pattern mirrors the macro scenario: power increases with both array size and

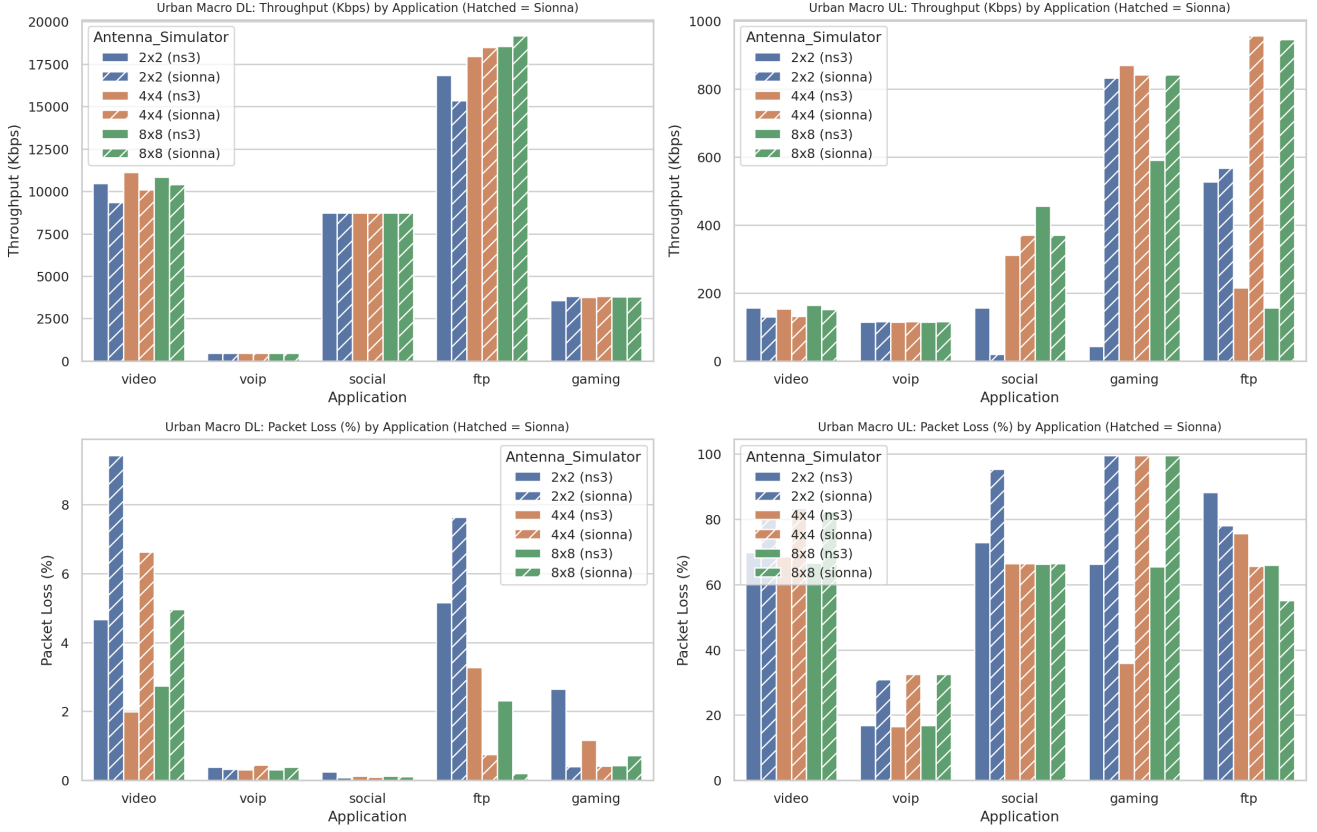


Figure 7: Urban macro application-level throughput and packet loss by application, direction, and gNB array size. Solid bars: ns-3; hatched bars: Sionna RT.

traffic load, and Sionna RT reports slightly lower consumption than ns-3 under high load, because its lower predicted channel quality leads to different resource-block allocations (Fig. 10). Under low load, both simulators report 23 W, 33 W, and 68 W for 2×2 , 4×4 , and 8×8 respectively. The urban micro scenario presents a notable high-load extreme: the 8×8 configuration reaches 215 W (ns-3) and 193 W (Sionna RT) — roughly three times the macro high-load figure — driven by the combined demand of a 400 MHz mmWave channel and dense multipath-induced retransmissions.

UE power consumption (0.05 W at low load, 0.09 W at high load) is nearly identical between the two simulators and independent of gNB array size, confirming that UEs operate at near-maximum transmit power against the challenging urban channel.

7. Conclusion

This paper presented **sionnart**: a pybind11-embedded integration of the Sionna RT ray tracing engine into ns-3, augmented by a displacement-based coherence cache and a proactive adaptive virtual receiver mechanism, both designed to keep per-step GPU overhead manageable. The embedded design achieves up to $43.8\times$ lower runtime than the two-process **ns3sionna**

baseline, maintains a GPU memory footprint below 500 MiB, and in stationary scenarios outperforms even the analytical Friis baseline owing to cache-driven elimination of repeated ray tracing calls. Applied to a 3GPP NR deployment across free-space, urban macro, and urban micro environments, the framework shows that a geometry-aware channel model produces more conservative — and physically more detailed — channel quality estimates than stochastic models, and confirms that 8×8 gNB antenna arrays are essential for overcoming harsh propagation conditions. As future work, the site-specific multipath data produced by **sionnart** will be used to extend the framework toward Integrated Sensing and Communications (ISAC) evaluation, enabling joint simulation of communication and sensing within a single ns-3 experiment.

References

- [1] 3GPP, “Study on Channel Model for Frequencies from 0.5 to 100 GHz,” 3GPP TR 38.901, v17.0.0, 2022.
- [2] A. M. Al-Samman, M. Cheffena, O. Elijah, Y. A. Al-Gumaei, S. K. Abdul Rahim, and T. A. Rahman, “Survey of Millimeter-Wave Propagation Measurements and Models in Indoor Environments,” *Electronics*, vol. 10, no. 14, p. 1653, 2021. doi: <https://doi.org/10.3390/electronics10141653>

Embedded Ray-Traced Propagation for ns-3

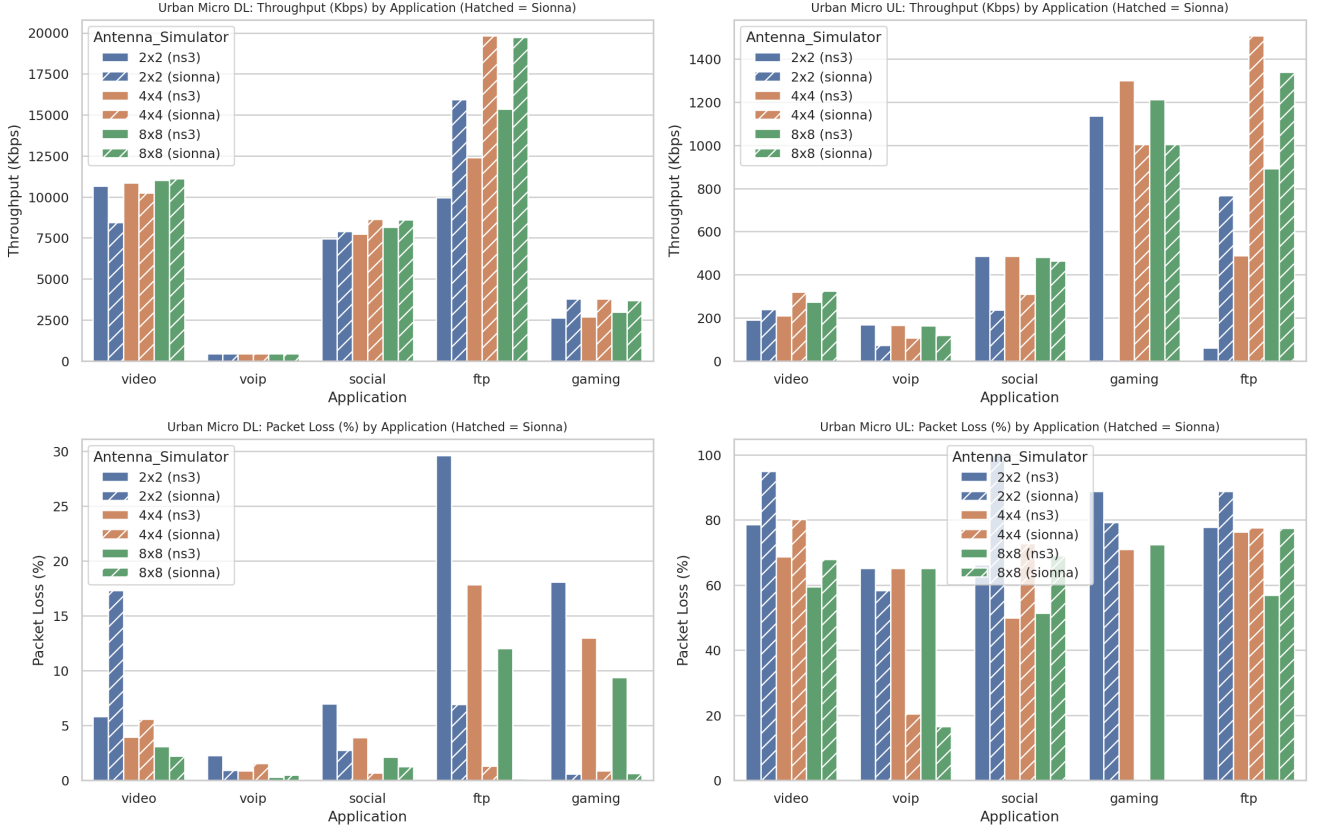


Figure 8: Urban micro application-level throughput and packet loss by application, direction, and gNB array size. Solid bars: ns-3; hatched bars: Sionna RT.

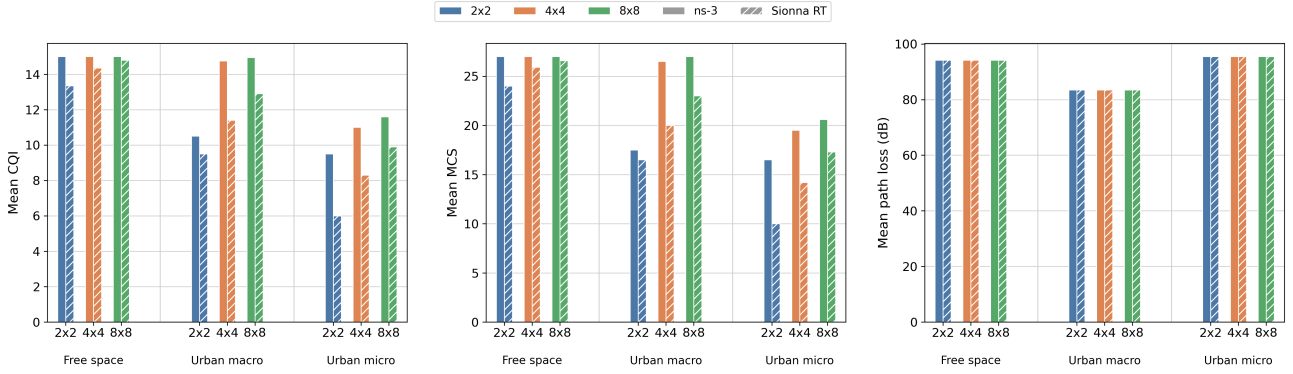


Figure 9: Propagation-related metrics (path loss, received power, path delay, CQI, MCS) across three radio environments and gNB array sizes, comparing ns-3 and Sionna RT under high traffic load.

- [3] C. Han, Y. Wang, and Y. Li, "Terahertz Wireless Channels: A Holistic Survey on Measurement, Modeling, and Analysis," *arXiv preprint arXiv:2111.04522*, 2021. doi: <https://doi.org/10.48550/arXiv.2111.04522>
- [4] W. Zhou, R. Zhang, G. Chen, and W. Wu, "Integrated Sensing and Communication Waveform Design: A Survey," *IEEE Open Journal of the Communications Society*, vol. 3, pp. 1930–1949, 2022. doi: <https://doi.org/10.1109/OJCOMS.2022.3215683>
- [5] Z. Gao, Z. Wan, D. Zheng, M. Matthaiou, and B. Ai, "Integrated Sensing and Communication With mmWave Massive MIMO: A Compressed Sampling Perspective," *IEEE Transactions on Wireless Communications*, vol. 22, no. 3, pp. 1745–1762, 2023. doi: <https://doi.org/10.1109/TWC.2022.3206614>
- [6] L. Cazzella, D. Tagliaferri, and M. Mizmizi, "Deep Learning of Transferable MIMO Channel Modes for 6G V2X Communications," *IEEE Transactions on Antennas and Propagation*, vol. 70, no. 6, pp. 4127–4139, 2022. doi: <https://doi.org/10.1109/TAP.2022.3156833>

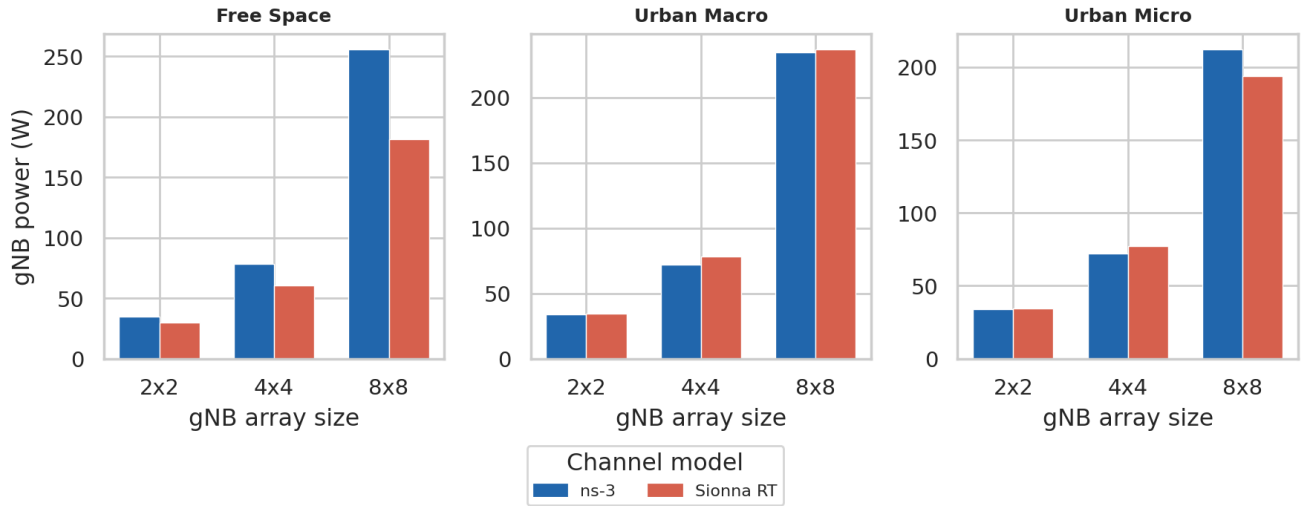


Figure 10: gNB power consumption (W) by array size and radio environment, comparing ns-3 and Sionna RT under high traffic load.

- //doi.org/10.1109/TAP.2022.3169950
- [7] B. Bojovic, Z. Ali, S. Lagen, and L. Giupponi, “ns-3 and 5G-LENA Extensions to Support Dual-Polarized MIMO,” in *Proceedings of the Workshop on ns-3*, 2022, pp. 1–9. doi: <https://doi.org/10.1145/3532577.3532595>
 - [8] The ns-3 Consortium, “ns-3 Network Simulator.” [Online]. Available: <https://www.nsnam.org/>
 - [9] N. Patriciello, S. Lagen, B. Bojovic, and L. Giupponi, “An E2E Simulator for 5G NR Networks,” *Simulation Modelling Practice and Theory*, vol. 96, p. 101933, 2019. doi: <https://doi.org/10.1016/j.simpat.2019.101933>
 - [10] S. Bastos, A. P. Oliveira, and D. T. Suzuki, “Generation of 5G/6G Wireless Channels Using Raymobtime with Sionna’s Ray-Tracing,” in *Proceedings of the Brazilian Telecommunications and Signal Processing Symposium*, 2023. doi: <https://doi.org/10.14209/sbirt.2023.1570923823>
 - [11] J. Hoydis *et al.*, “Sionna: An Open-Source Library for Next-Generation Physical Layer Research,” NVIDIA Research. [Online]. Available: <https://nvlabs.github.io/sionna/>
 - [12] F. Ait Aoudia, J. Hoydis, M. Nimier-David *et al.*, “Sionna RT: Technical Report,” *arXiv preprint arXiv:2504.21719*, 2025. doi: <https://doi.org/10.48550/arXiv.2504.21719>
 - [13] A. Zubow, Y. Pilz, S. Rösler, and F. Dressler, “Ns3 Meets Sionna: Using Realistic Channels in Network Simulation,” *arXiv preprint arXiv:2412.20524*, 2024. doi: <https://doi.org/10.48550/arXiv.2412.20524>
 - [14] W. Jakob, J. Rhineland, and D. Moldovan, “pybind11 – Seamless Operability between C++11 and Python.” [Online]. Available: <https://github.com/pybind/pybind11>
 - [15] A. Varga and R. Hornig, “An Overview of the OMNeT++ Simulation Environment,” in *Proceedings of the 1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems (SIMUTools)*, 2008.
 - [16] G. Nardini, D. Sabella, G. Stea, P. Thakkar, and A. Virdis, “Simu5G–An OMNeT++ Library for End-to-End Performance Evaluation of 5G Networks,” *IEEE Access*, vol. 8, pp. 181176–181191, 2020. doi: <https://doi.org/10.1109/ACCESS.2020.3028550>
 - [17] P. Mogensen *et al.*, “LTE Capacity Compared to the Shannon Bound,” in *Proceedings of the IEEE Vehicular Technology Conference (VTC)*, 2007.
 - [18] B. Bojovic, S. Lagen, and L. Giupponi, “Realistic Beamforming Design Using SRS-Based Channel Estimate for ns-3 5G-LENA Module,” in *Proceedings of the Workshop on ns-3*, 2021, pp. 81–87. doi: <https://doi.org/10.1145/3460797.3460809>
 - [19] A. N. Uwaechia and N. M. Mahyuddin, “A Comprehensive Survey on Millimeter Wave Communications for Fifth-Generation Wireless Networks: Feasibility and Challenges,” *IEEE Access*, vol. 8, pp. 62367–62414, 2020. doi: <https://doi.org/10.1109/ACCESS.2020.2984204>
 - [20] H. Hua, J. Xu, and T. X. Han, “Optimal Transmit Beamforming for Integrated Sensing and Communication,” *arXiv preprint arXiv:2104.11871*, 2021. doi: <https://doi.org/10.48550/arXiv.2104.11871>
 - [21] A. Samorzewski, M. Deruyck, and A. Kliks, “Energy Consumption in RES-Aware 5G Networks,” in *Proceedings of IEEE Global Communications Conference (GLOBECOM)*, 2023, pp. 1024–1029. doi: <https://doi.org/10.1109/GLOBECOM54140.2023.10437451>
 - [22] Ericsson, “Ericsson Mobility Report,” Nov. 2025. [Online]. Available: <https://www.ericsson.com/4aca6f/assets/local/reports-papers/mobility-report/documents/2025/ericsson-mobility-report-november-2025.pdf>